PGR211: Advanced Programming for Data Science
Prof. Dr. Rashmi Gupta
Kristiania University of Applied Sciences, Oslo, Norway

Final Exam

- This is a written home examination completed in groups of 2-3 students. You will design, implement, document, and deploy a Django web application. The assessment reflects the programming and Django workflow practised during the course, including programming paradigms, models, migrations, admin, forms, validation, views, URLs, templates, CRUD operations, and production deployment.
    - **Submit the complete Django project source code as a ZIP file.**
    - **Submit a code explanation/report containing (not any word/page limit):**
        - A concise **explanation** of the project structure and the main scripts, including how OOP and functional-programming concepts are used where appropriate.
        - **A working deployment link** and a short explanation of the production configuration and deployment process. The deployed application must remain accessible during assessment.
- **Duration:** 4 weeks.
- **Grading scale:** The Norwegian grading system uses the graded scale A - F, where A is the best grade, E is the lowest pass grade, and F is fail
- **Weighting:** 100% of the overall grade
- Start uploading your exam paper/file ahead of time, as it may take a long time to upload.
- Exam papers that are not handed in on Wiseflow by the specified time of the submission date will not proceed to assessment. No late solutions will be accepted. Your submitted documents will be checked for plagiarism.
- Before submitting, remember to check that all files can be opened and that every file is included. It may be a good idea to check the saved files on several machines before submitting them in Wiseflow.
- Incorrect file format or lacking documents may result in the submission not being passed or assessed.
- Show your work, and be creative, as partial credit will be given. You will be graded not only on the correctness of your answer but also on the clarity with which you express it.

Attachment: Grading Scale
The table below shows how the Universities Norway council (UHR) defines general, qualitative requirements for the corresponding letter grades. The criteria below should be used as a guideline in assigning grades for examinations unless other assessment criteria are explicitly provided for a particular assignment.

Letter grade	Descriptor	General, not subject-specific, description of assessment criteria
A	Outstanding	**Outstanding performance that clearly stands out.** The solution demonstrates excellent command of Python programming concepts, Django development, database-backed web application design, and a robust, working production deployment with clear technical documentation.
B	Very good	**Very strong performance.** The candidate demonstrates very good knowledge of the learning outcomes, including Django development and successful deployment.
C	Good	**A good performance that satisfies most assessment criteria.** The candidate displays a good knowledge in the most important areas of assessment.
D	Fair	**An acceptable performance with some clear deficiencies**. The candidate somewhat displays a level of sound knowledge of learnt programming concepts.
E	Sufficient	**A performance that only satisfies the minimum requirements.** The candidate displays a poor level of knowledge of learnt programming concepts.
F	Fail	**A performance that does not satisfy the most basic formal requirements.** The candidate lacks sound knowledge of learnt programming concepts.

Point total and grading scale [1]

Letter Grade	Corresponding numerical grade	Westerdals Grading scale
A	90-100	93 -100
B	80 – 89	78 – 92
C	60 - 79	59 - 77
D	50 - 59	51 - 58
E	40 - 49	40 - 50
F	0 - 39	0 - 39

Good luck!

Exam: Building and Deploying a Vlog Application with Django

Instructions: Develop the application as a complete, working Django project. Your submission must include the source code, a concise technical report with relevant code snippets/screenshots, and a working deployed URL. Explain important design choices rather than only pasting code.

Total: 100 points.

Section 1: Project Architecture and Programming Concepts (10 Points)

1. Project structure and Django architecture (5 Points)

- Present the main project/app structure and explain the purpose of key files such as settings.py, urls.py, models.py, views.py, forms.py, admin.py, and templates.
- Briefly explain the request -> URL -> view -> model/form -> template -> response flow.

2. Programming concepts used in the project (5 Points)

- Identify and explain at least one meaningful use of an OOP concept such as classes, inheritance, encapsulation, or polymorphism.
- Show one small, appropriate use of a functional-programming technique such as a pure helper function, lambda/key function, comprehension, map/filter, or another higher-order function, and explain why it is suitable.

Section 2: Models, Database, Migrations and Admin (15 Points)

1. Design the data model (8 Points)

- Create a VlogPost model containing suitable fields such as title, video_url, description, author, published_date, tags/category, and any additional field(s) that improve the application.
- Choose appropriate Django field types, constraints/defaults, and implement a useful __str__() representation. Explain your choices.

2. Database workflow and admin interface (7 Points)

- Create and apply migrations and explain the role of makemigrations and migrate.
- Register the model(s) in Django Admin and customise the list view using suitable features such as list_display, search_fields, list_filter, and ordering.
- Include evidence that records can be created, updated, and inspected in the database/admin interface.

Section 3: Forms, ModelForms, Validation and CSRF (15 Points)

1. Create a ModelForm for VlogPost (8 Points)

- Create VlogPostForm using forms.ModelForm and include appropriate fields for creating/editing a vlog.
- Render the form in a template and include {% csrf_token %} for POST submission.
- Demonstrate form.is_valid(), validation/error display, and form.save() where appropriate.

2. Explain the three form approaches (7 Points)

- Briefly compare a manually written HTML form, forms.Form, and forms.ModelForm. State when each approach may be useful.
- Add at least one meaningful validation rule or user-friendly widget/label/placeholder and explain it.

Section 4: Views, URLs, Templates and CRUD (25 Points)

1. Implement a complete CRUD workflow (15 Points)

- List: display all vlog posts, with a clear ordering and optional pagination.
- Detail: display one vlog post using its id/pk.
- Create: add a new vlog through the ModelForm.
- Update: edit an existing vlog.
- Delete: delete a vlog safely using a confirmation step.
- You may use function-based or class-based views, but your report must explain how requests and responses are handled.

2. URL configuration and templates (10 Points)

- Define clear URL patterns for list, detail, create, update, and delete operations.
- Use template inheritance (for example, base.html) and display model/form data using Django Template Language.
- Use static files (CSS and/or images) to create a clear, usable interface.
- Demonstrate correct GET/POST handling and an appropriate redirect after successful form submission.

Section 5: Deployment and Production Readiness (30 Points)

1. Prepare the application for deployment (10 Points)

- Create and verify the dependency file (for example, requirements.txt) and all configuration files required by the deployment platform.
- Separate development and production-sensitive settings. Do not expose passwords, API keys, database credentials, or SECRET_KEY in submitted public code/repositories.
- Configure production-relevant Django settings correctly, including DEBUG, ALLOWED_HOSTS, static files, and any host/origin settings required by the platform.

2. Deploy the working application (12 Points)

- Deploy the Django application to AWS Elastic Beanstalk that is approved for this course.
- Run/apply database migrations in the deployed environment and ensure static assets are served correctly.
- Provide a working URL that the examiner can open without local setup. Core CRUD functionality must work on the deployed version.

3. Deployment evidence and technical explanation (8 Points)

- Include concise screenshots/evidence of deployment configuration/build/runtime status and the live application.
- Explain the main deployment steps from local project to live service, including environment variables and production settings.
- Document at least one deployment problem encountered (or a realistic deployment risk) and explain how it was diagnosed/resolved or mitigated.

Section 6: Documentation, Verification and Submission Quality (5 Points)

1. Provide a concise report that connects your implementation to the course concepts, includes the deployed URL, and clearly identifies where the examiner can find the required functionality. (3 Points)

2. Before submission, verify that the ZIP opens correctly, the project can be understood from the submitted files, and the deployed application is reachable and functional. (2 Points)